Project Template

NLP Emotions Classification

Project Information

Project Title:

Text Emotion Classification using NLP

Team Members:

- Student 1: [Nada Hossam (235319)]
- Student 2: [Rhama Ahmed(235478)]
- Student 3: [Mohamed Alaa(235315)]
- Student 4: [Ahmed Reda(235228)]

Teacher Assistant: [Salma Abedlmonem]

Date: 10 May 2026

Milestone 1: Data Collection & Preprocessing

1. Data Collection

Dataset: dair-ai/emotion (Hugging Face Datasets)

The dataset was loaded using the Hugging Face datasets library and consists of English Twitter messages labeled with one of six emotions: sadness, joy, love, anger, fear, and surprise.

Split	Samples	Description
Train	16,000	Used for model training
Validation	2,000	Used for hyperparameter tuning
Test	2,000	Used for final evaluation

2. Data Exploration (EDA)

The following exploratory analyses were performed:

- Emotion Distribution: Plotted a count plot showing the distribution of each emotion label across the training split.
- Text Length Analysis: Computed average (≈ 67 chars), maximum, and minimum text lengths; visualized via histogram and box plot per emotion.
- Class Distribution per Split: Bar charts comparing label counts across train, validation, and test splits confirmed balanced class representation.
- Word Clouds: Generated word clouds for each of the six emotion classes to visualize the most frequent terms per class.
- Sample Texts: Printed a sample text from each emotion to qualitatively understand the data.

3. Data Preprocessing

A clean_text() function was applied to all training samples:

- Removed non-alphabetical characters using regex (re.sub).
- Converted all text to lowercase.
- Removed punctuation via str.maketrans.
- Tokenized text with NLTK word_tokenize.
- Removed English stop words using NLTK stopwords corpus.
- Applied WordNet lemmatization to reduce words to their base form.

Duplicates were detected and removed from the training set. The final cleaned text was stored in a new column cleaned_text.

Milestone 2: Text Preprocessing & Feature Engineering

1. Text Cleaning

As described in Milestone 1, a unified clean_text() function handles all text normalization steps: regex filtering, lowercasing, stop-word removal, and lemmatization. A separate clean_for_lstm() function was created for the LSTM pipeline, following the same steps but operating on the Hugging Face dataset splits directly.

2. Vectorization

TF-IDF Vectorization (for ML classifiers):

- Used TfidfVectorizer with max_features=5000.
- N-gram range: (1, 3) — unigrams, bigrams, and trigrams.

- sublinear_tf=True applied to dampen high-frequency terms.
- Strip accents (unicode) and English stop words enabled.

Sequence Tokenization (for LSTM):

- Keras Tokenizer with vocab_size=20,000 and <OOV> token.
- Sequences padded to max_len=100 using pre-padding.
- Labels one-hot encoded using to_categorical.

3. Feature Engineering

For ML models: TF-IDF features (5,000-dimensional sparse vectors) served as input features.

For LSTM: Embedded representations via a trainable Embedding layer (embed_dim=128) learned contextual word representations during training.

Label encoding used id2label / label2id mappings derived directly from the dataset's feature schema, ensuring consistency across all splits.

Milestone 3: Model Development

1. Model Selection

Two families of models were evaluated:

Machine Learning Classifiers (with TF-IDF features):

- Multinomial Naive Bayes
- Logistic Regression
- Random Forest
- Support Vector Machine (SVC)
- SGD Classifier (SVM alternative with hinge loss)
- K-Nearest Neighbors (cosine metric)

Deep Learning Model:

- LSTM (Long Short-Term Memory) neural network

2. Model Training

ML Models: All six classifiers were trained on X_train_tfidf and evaluated on X_test_tfidf.

LSTM Model Architecture:

- Embedding Layer (vocab=20,000, dim=128, input_len=100)
- SpatialDropout1D (0.3)
- LSTM Layer (128 units, dropout=0.3, recurrent_dropout=0.2)
- BatchNormalization
- Dense Layer (128 units, ReLU activation)
- Dropout (0.5)
- Output Dense Layer (6 units, Softmax activation)

LSTM Training config: Adam optimizer, categorical cross-entropy loss, batch_size=64, up to 20 epochs with EarlyStopping (patience=2, monitor=val_loss).

3. Model Evaluation

All models were evaluated on accuracy and weighted F1-score on the held-out test set. Confusion matrices (counts and normalized) were plotted for the best ML model.

Model	Test Accuracy	F1 (Weighted)	Train Time
Logistic Regression	~0.89	~0.89	~2s
SVM (SGD)	~0.87	~0.87	~3s
Random Forest	~0.84	~0.84	~30s
Multinomial NB	~0.83	~0.83	~0.1s
KNN	~0.78	~0.78	~1s
LSTM (Deep)	~0.92	~0.92	~60s

4. Final Model Selection

The LSTM model achieved the highest test accuracy (~92%) and weighted F1-score, outperforming all ML classifiers. Among traditional ML models, Logistic Regression was the best performer with ~89% accuracy and F1 score.

The final selected model for deployment is the LSTM model, saved as deep_bilstm_emotion.keras, with its tokenizer saved separately as lstm_tokenizer.pkl.

Milestone 4: Deployment

1. Deployment

All trained models were serialized for deployment:

ML Models (saved via pickle):

- logistic_regression.pkl
- random_forest.pkl
- svm_model.pkl
- gradient_boosting.pkl
- multinomial_nb.pkl
- knn_model.pkl
- tfidf_vectorizer.pkl — TF-IDF vectorizer
- id2label.pkl — Label mapping dictionary

LSTM Model (saved via Keras):

- deep_bilstm_emotion.keras — Full Keras model with weights
- lstm_tokenizer.pkl — Keras tokenizer
- id2label.pkl — Shared label mapping

A predict_emotion() function (ML) and predict_emotion_lstm() function (LSTM) were implemented to accept raw input text and return the predicted emotion label along with confidence scores and per-class probabilities.

Milestone 5: Final Report

1. Project Summary

This project developed a text-based emotion classification system using NLP techniques on the dair-ai/emotion dataset. The system classifies English text into six emotions: sadness, joy, love, anger, fear, and surprise. Two parallel pipelines were implemented: a traditional ML pipeline using TF-IDF features with six classifiers, and a deep learning pipeline using an LSTM neural network. The LSTM model achieved ~92% accuracy, outperforming all ML baselines.

2. Business Impact

Emotion classification has broad real-world applications:

- Customer Feedback Analysis: Automatically detect customer sentiment in support tickets and reviews to prioritize urgent negative feedback.
- Mental Health Monitoring: Identify distress signals in user-generated text for early intervention platforms.

- Social Media Analytics: Monitor emotional trends during events, product launches, or crises.
- Chatbot Enhancement: Enable conversational agents to respond empathetically based on detected user emotion.

3. Challenges Faced

- Class Imbalance: The dataset had imbalanced emotion classes, which required careful monitoring of per-class F1-scores.
- Text Ambiguity: Short social media texts often lack context, making certain emotions (e.g., fear vs. sadness) difficult to distinguish.
- Feature Engineering Trade-offs: TF-IDF loses word order; the LSTM better captures sequential context but requires more computational resources.
- LSTM Training Time: Deep learning training was significantly slower than traditional ML models, requiring EarlyStopping to prevent overfitting.

4. Future Improvements

- Transformer Models: Fine-tune pre-trained models such as BERT or RoBERTa for potentially higher accuracy.
- Data Augmentation: Apply techniques like back-translation or synonym replacement to balance underrepresented emotion classes.
- Multi-label Classification: Extend the system to detect mixed emotions within a single text.
- Real-time API: Wrap the inference pipeline in a REST API (e.g., FastAPI/Flask) for production deployment.
- Explainability: Integrate LIME or SHAP to provide interpretable explanations for model predictions.

Presentation Section

Demo Description:

The live demo showcases the emotion classification system accepting raw English text and returning the predicted emotion label with confidence score. Two modes are demonstrated:

- ML Mode: Input text is cleaned, TF-IDF vectorized, and classified by Logistic Regression in real-time.
- LSTM Mode: Input text is tokenized, padded, and passed through the deep LSTM model, returning the predicted emotion with top-3 class probabilities and emoji indicators.

Sample predictions from both pipelines are shown side-by-side to illustrate the performance difference between traditional ML and deep learning approaches.